SRM University – AP, Andhra Pradesh

Neeru Konda, Mangalagiri Mandal, Guntur District – 522240

Blockchain Technologies

Comprehensive Study Notes

Course Code: AML 571

Course Category: Core Elective (CW)

L – T – P – C: 3 – 0 – 0 – 3

Department: Computer Science and Engineering

Compiled for: Venkat Rayudu Alapati (AP25122040029)

Programme: M.Tech (AI & ML)

Academic Year: 2025 – 2026

Unit 1

Cryptographic Foundations and Distributed Systems

Required Contact Hours: **9**

- Introduction to Blockchain — What and Why
- Cryptographic hash functions (SHA-256, Keccak) and properties
- Merkle trees, Patricia tries, Verkle trees, KZG commitments
- Digital signatures: ECDSA, EdDSA, Schnorr, BLS
- Distributed systems: CAP/FLP, Byzantine Generals, 3f+1 bound

The chapters that follow expand each topic above with theory, worked examples, reading lists, hands-on lab guidance, and end-of-chapter self-assessment questions.

Blockchain Module 1 — Cryptographic Foundations

> ***Goal**: master the four cryptographic primitives every blockchain rests on: hash functions, Merkle trees, digital signatures, and commitment schemes. Without these, every later concept is hand-waving.*

1.1 Why cryptography is the load-bearing pillar

A blockchain is, mechanically, a **replicated state machine** in an adversarial network. Cryptography gives us three guarantees we cannot get any other way:

1. **Integrity** — a hash chain lets every honest node detect tampering.
2. **Authentication** — digital signatures bind actions to identities (public keys).
3. **Commitment** — once published, a value cannot be silently changed.

Everything else (consensus, smart contracts, governance) is engineering on top of those three primitives.

1.2 Cryptographic hash functions

A hash function $H: \{0,1\}^* \to \{0,1\}^n$ (typically n=256) is used to compress arbitrary data to a fixed-length digest.

Required properties

Property	Definition	Why it matters
Pre-image resistance	Given y, infeasible to find x s.t. $H(x)=y$	Hides inputs (commitments, password storage)
Second pre-image resistance	Given x, infeasible to find $x' \neq x$ s.t. $H(x') = H(x)$	Prevents targeted tampering
Collision resistance	Infeasible to find any (x, x') with $H(x)=H(x')$	Prevents two valid blocks colliding
Puzzle-friendliness	For any k, the distribution of $H(k\|x)$ is "random enough" that you must brute-force to find solutions	Foundation of Proof-of-Work

Standard choice: **SHA-256** for Bitcoin; **Keccak-256** for Ethereum; **BLAKE3** is faster but less established.

Birthday bound

Collision attacks on an n-bit hash take roughly $2^{n/2}$ work. So SHA-256 gives ~128-bit collision security — currently safe.

Hash chain

```
block_0   block_1   block_2
[h_prev=⊥] → [h_prev = H(block_0)] → [h_prev = H(block_1)] → ...
```

Any tampered block changes its hash, breaking every subsequent link. This is the simplest data structure with **tamper evidence**.

1.3 Merkle trees

To prove that one transaction is included in a block without sending the whole block, we use a **Merkle tree** (a.k.a. hash tree).

```
          root = H(H1‖H2)
          /               \
    H1=H(A‖B)          H2=H(C‖D)
     /     \             /     \
  H(A)     H(B)       H(C)     H(D)
```

Inclusion proof for transaction `C`: send `H(D)` and `H1` (log n hashes). The verifier recomputes:

```
H2' = H(H(C) ‖ H(D))
root' = H(H1 ‖ H2')
```

If `root' == published root`, `C` is in the tree.

Real-world variants

- **Bitcoin Merkle tree** — flat binary tree of transactions in a block.
- **Patricia/Verkle tries** (Ethereum) — Merkle trie indexed by account address; supports efficient state proofs.
- **Sparse Merkle trees** — used in Filecoin, rollups.
- **Verkle trees** — replace hashes with vector commitments (e.g., KZG) — proof size O(1) instead of O(log n). Important for stateless Ethereum.

Research angle

Verkle trees, IVC (incrementally verifiable computation), and "history expiry" are active 2023–2026 research directions; see EIP-6800.

1.4 Digital signatures

Signature scheme = `(KeyGen, Sign, Verify)`.

Scheme	Curve / basis	Used by
ECDSA	secp256k1	Bitcoin, Ethereum (pre-account abstraction)
EdDSA / Ed25519	edwards25519	Solana, Cardano, Tezos
Schnorr (BIP-340)	secp256k1	Bitcoin Taproot, supports key aggregation
BLS	BLS12-381	Ethereum consensus (validator aggregation)

Why BLS matters

A BLS signature scheme allows you to *aggregate* `n` signatures from `n` validators into a single 96-byte signature that verifies all of them at once. Without this, Ethereum's PoS could not scale to 1M+ validators.

Public keys are addresses

In Bitcoin: `address = Base58Check(RIPEMD160(SHA256(pubkey)))`. In Ethereum: `address = last 20 bytes of Keccak256(pubkey)`.

This is how a string of letters and numbers binds to a specific cryptographic identity.

1.5 Commitment schemes

A **commitment** lets you publish a binding fingerprint of a value `v` without revealing it (hiding), and later reveal `v` such that nobody can dispute it (binding).

Simplest commitment: `c = H(v ‖ r)` where `r` is a random nonce.

Pedersen commitments

`c = v·G + r·H` where G, H are generators of an elliptic curve group with unknown discrete log relation. Pedersen is **homomorphic** — `Commit(v1) + Commit(v2) = Commit(v1+v2)` — which makes it the basis of confidential transactions (Monero, Mimblewimble).

KZG / polynomial commitments

A KZG commitment lets you commit to a polynomial `p(x)` and later open it at any point with a constant-size proof. KZG underlies:

- Ethereum's **EIP-4844 blob transactions** (proto-danksharding)
- Verkle trees
- PLONK-family zk-SNARKs

This is *the* commitment scheme of the modern blockchain stack.

1.6 Putting it all together — a toy block

```
class Block:
    prev_hash: bytes              # 32 bytes from previous block
    merkle_root: bytes            # root over transactions
    timestamp: int
    nonce: int                    # PoW puzzle solution
    transactions: list[Tx]

    def hash(self) -> bytes:
        return sha256(prev_hash + merkle_root + ts + nonce)

# Each Tx contains: from_pubkey, to_pubkey, amount, signature
# Tx is valid iff verify(from_pubkey, msg=(from,to,amount), signature)
```

This is — quite literally — Bitcoin minus the consensus rules.

1.7 Reading list

- Narayanan et al., *Bitcoin and Cryptocurrency Technologies*, Ch 1.
- Katz & Lindell, *Introduction to Modern Cryptography*, Ch 5 (hash) & Ch 12 (signatures).
- Boneh & Shoup, *A Graduate Course in Applied Cryptography* (free online).
- Boneh, Drake, Fisch, Gabizon — *KZG polynomial commitments and applications* (talks).
- EIP-4844 specification (proto-danksharding).

1.8 Lab (2 hrs)

Implement, in Python with `hashlib` and `ecdsa` only (no blockchain framework):

1. A `Block` class with header + Merkle root.
2. A `Wallet` class that generates `(privkey, pubkey, address)`.
3. A `Transaction` class with signing + verification.
4. A 5-block chain — show that flipping a bit in block 2 breaks all subsequent hashes.

1.9 Quiz

1. Why is collision resistance strictly stronger than second-preimage resistance?
2. What is the size (in hashes) of a Merkle proof for one of 1,048,576 leaves?
3. Why does BLS signature aggregation matter for Proof-of-Stake?
4. State one property that Pedersen has and SHA-based commitments don't.
5. Roughly how many SHA-256 evaluations are needed to find a collision?

Blockchain Module 2 — Distributed Systems & the CAP/FLP Walls

> ***Goal**: Understand why "just add a database" doesn't work in an open, adversarial network. Build the intuition for why blockchains are slow.*

2.1 The classical distributed systems triangle

Before blockchains, the problem of "n computers agreeing on the same state" was studied for decades. Two impossibility results define the playing field.

CAP theorem (Brewer, 2000; formalized Gilbert & Lynch, 2002)

In a distributed system you can pick at most **two** of:

- **C — Consistency** (every read gets the latest write)
- **A — Availability** (every request gets a non-error response)
- **P — Partition tolerance** (system continues operating across network splits)

Real networks **always** drop packets, so P is mandatory. The real choice is **CP vs AP**.

System	Choice
PostgreSQL replica	CP
Cassandra	AP
Bitcoin (eventual finality)	AP-ish
Ethereum PoS (with finality gadget)	CP for finalized, AP for tip

FLP impossibility (Fischer, Lynch, Paterson, 1985)

In a purely **asynchronous** system, no deterministic protocol can guarantee consensus if even one node may crash.

This is why every real-world consensus protocol assumes one of:

- **Partial synchrony** (messages eventually arrive within Δ) — used by PBFT, HotStuff, Tendermint, Casper.
- **Randomization** (Ben-Or, common coin) — used by Algorand, HoneyBadger BFT.
- **Synchrony** (bounded message delay) — used by classical Byzantine agreement.

You cannot escape FLP — you can only choose which assumption to weaken.

2.2 Failure models

Model	What can a node do?	Examples

Crash-stop	Halt and stop responding	Paxos, Raft
Crash-recovery	Stop and later restart	Real distributed DBs
Byzantine	Arbitrary, malicious behavior	Bitcoin, Ethereum, PBFT
Rational	Self-interested, follows protocol when incentivized	BAR-Tolerant Replication, blockchain proper

Blockchain's contribution to distributed systems theory was demonstrating **economic-rational Byzantine tolerance at planetary scale**.

2.3 The Byzantine Generals Problem (Lamport, Shostak, Pease, 1982)

`n` generals, `f` of whom are traitors. They must agree on attack/retreat using only messages.

Classical bound: agreement is possible iff `n ≥ 3f + 1` for synchronous, signed-message-free protocols. With signatures (authenticated Byzantine), `n ≥ 2f + 1` suffices but requires synchrony.

The number `2/3` shows up everywhere in BFT:

- Tendermint needs >2/3 honest stake to finalize.
- HotStuff (used by Diem, Aptos) — same.
- Ethereum PoS finality — needs 2/3 honest validators.

2.4 Replicated state machines (RSM)

A blockchain *is* a replicated state machine.

```
state_(t+1) = transition(state_t, tx)
```

For all honest nodes to agree on `state_t`, they must agree on:

1. The **set** of transactions applied
2. The **order** of transactions applied

Consensus protocols solve problem #2. Problem #1 is solved trivially once order is fixed (everyone broadcasts and applies in agreed order).

This framing shows that *consensus is about ordering, not about validity*. A transaction can be ordered before being validated.

2.5 Leader-based vs leaderless consensus

Family	Mechanism	Example
Leader-based	One node proposes a block per round	PBFT, HotStuff, Raft, Tendermint
Probabilistic leader	Random leader per round (VRF)	Algorand, Cardano Ouroboros

Leaderless	DAG of blocks, no single proposer	IOTA Tangle, Aleph, Narwhal-Bullshark

Leader-based is simpler but creates a bottleneck. Leaderless / DAG-based protocols (Narwhal, Bullshark, Mysticeti from Sui; Aleph from Aleph Zero) are the 2023–2026 frontier and achieve >100k TPS in published benchmarks.

2.6 The fork problem & finality

Fork = the chain splits because two nodes propose conflicting blocks.

Finality type	Definition	Example
Probabilistic	Probability of reversal → 0 as more blocks confirm	Bitcoin (6 blocks ≈ 99.99%)
Economic	Reversal would cost >X tokens	PoS slashing
Absolute	Finalized block cannot be reverted without > f Byzantine breach	Tendermint, PBFT-style

Bitcoin: probabilistic, eventual. Ethereum post-Merge: hybrid — tip is probabilistic, finalized checkpoint is absolute (with 1/3 of stake at risk to revert).

2.7 Why throughput is limited

A back-of-envelope for any synchronous BFT protocol:

```
throughput ≤ block_size / (network_delay + signature_verification + state_writes)
```

If you increase block size, propagation slows → forks. This is **Nakamoto's tradeoff**.

L2s / sharding / DAG mempools are all attempts to shift parts of this equation off the critical path.

2.8 The mempool

Before consensus, transactions live in each node's **mempool** (memory pool). Selection from mempool into a block is where **MEV** (Maximal Extractable Value) lives. Most blockchain economic exploits originate here.

2.9 Reading list

- Lamport, Shostak, Pease — *The Byzantine Generals Problem*, 1982.
- Fischer, Lynch, Paterson — *Impossibility of distributed consensus with one faulty process*, 1985.
- Castro & Liskov — *Practical Byzantine Fault Tolerance*, OSDI 1999.
- Yin et al. — *HotStuff: BFT consensus in the lens of blockchain*, PODC 2019.
- Garay, Kiayias, Leonardos — *The Bitcoin Backbone Protocol*, Eurocrypt 2015.
- Buchman, Kwon, Milosevic — *The latest gossip on BFT consensus* (Tendermint).

2.10 Lab (2 hrs)

Build a simulator (no real network) of 7 nodes running a toy 2/3-majority commit protocol:

1. Each round one leader proposes.
2. Each node either votes commit or aborts.
3. Crash 2 of the 7 → it should still finalize. Make 3 Byzantine → it must fail.
4. Plot finality latency vs. number of Byzantine nodes.

2.11 Quiz

1. Why does CAP force most blockchains to favor AP at the tip and CP at finality?
2. State the `3f+1` bound and why it appears in PBFT and Tendermint.
3. What's the difference between probabilistic and absolute finality?
4. Why does increasing block size hurt decentralization?
5. Name two DAG-based consensus protocols and one tradeoff they make vs leader-based ones.

Unit 2

Bitcoin and Ethereum

Required Contact Hours: **8**

- Bitcoin — UTXO model, Script, Proof-of-Work
- Difficulty adjustment, halving, selfish mining, 51% attack
- Ethereum — Account model, EVM, Gas mechanics
- Solidity, ERC standards (ERC-20, ERC-721, ERC-4337)
- Lightning Network and payment channels

The chapters that follow expand each topic above with theory, worked examples, reading lists, hands-on lab guidance, and end-of-chapter self-assessment questions.

Blockchain Module 3 — Bitcoin Deep Dive

> ***Goal**: Understand Bitcoin not as a coin but as the first working solution to Byzantine consensus over an open membership network. Every later blockchain is a delta against Bitcoin.*

3.1 The white paper in one paragraph

Satoshi Nakamoto (2008) proposed: combine `(a) hash-chained blocks, (b) PoW puzzle, (c) longest-chain rule, (d) economic incentives` so that **honest nodes following the protocol find it more profitable than attacking it**. The protocol is open — anyone can join — yet Byzantine-tolerant under the assumption that >50% of compute is honest.

3.2 The UTXO model

Bitcoin's state is not a list of balances. It's a set of **Unspent Transaction Outputs**.

```
Tx {
  inputs:  [ (prev_tx_id, output_index, signature) ... ]
  outputs: [ (amount, locking_script) ... ]
}
```

To spend, you reference previous outputs and prove you can unlock them (via Script). Total input value ≥ total output value; the difference is the miner fee.

Why UTXO?

- **Stateless validation** — verifying a tx needs only the referenced UTXOs, not full chain state.
- **Parallelizable** — txs touching disjoint UTXOs can be validated in parallel.
- **Privacy** — each UTXO can be a fresh address.

Ethereum chose the **account model** instead — simpler smart contracts but harder to parallelize.

3.3 Bitcoin Script

A stack-based, non-Turing-complete language. Standard "P2PKH" locking script:

```
OP_DUP OP_HASH160 <pubKeyHash> OP_EQUALVERIFY OP_CHECKSIG
```

Unlocking script supplies `<signature> <pubKey>`. Combined, the script evaluates to TRUE iff the signature is valid for that pubKey, whose hash matches `<pubKeyHash>`.

Why not Turing complete?

To bound execution time and prevent halting-problem attacks. Bitcoin chose this conservatism deliberately. Taproot

(2021) and Miniscript expanded expressiveness while keeping safety guarantees.

3.4 Proof-of-Work

The puzzle: find a nonce such that `H(block_header) < target`, where `target` is set so that the global network produces one block every 10 minutes.

```
while sha256(sha256(header_with(nonce))) >= target:
    nonce += 1
```

Difficulty adjustment

Every 2016 blocks (~2 weeks), retarget so that the previous 2016 blocks took ~2 weeks of wall time. This is what gives Bitcoin its stable block interval despite enormous hashrate growth.

Nakamoto's security argument

If an attacker has fraction `q < 0.5` of hashrate and is `z` blocks behind, the probability they ever catch up follows a Negative Binomial. By `z=6`, P(reversal) ≈ 0.001 for `q=0.1` and falls exponentially.

This is **probabilistic finality** and the source of the "wait 6 confirmations" folklore.

3.5 Block structure

```
Header (80 bytes):
  version, prev_hash, merkle_root, timestamp, bits (target), nonce
Body:
  varint count, then transactions
```

Maximum block size: 1MB (4 MB weight after SegWit). Average tx size: ~250 bytes → ~3.5 tx/sec theoretical max throughput. **This is the famous bottleneck**.

3.6 Mining and the incentive layer

Miner reward per block = `subsidy + transaction fees`.

Era	Block subsidy
2009–2012	50 BTC
2012–2016	25 BTC
2016–2020	12.5 BTC
2020–2024	6.25 BTC
2024–2028	3.125 BTC

...	halves every 210,000 blocks (~4 yrs)

After ~2140, subsidy → 0. Bitcoin's long-term security depends on fees alone — an open research question.

Selfish mining (Eyal & Sirer, 2014)

A miner with >25% hashrate can withhold blocks and gain disproportionate reward — proving Nakamoto's "honest-majority" assumption was naive. Mitigations are limited; this remains the canonical critique.

3.7 The 51% attack

With >50% hashrate, an attacker can:

- Double-spend (rewrite history they were a party to)
- Censor transactions
- **Cannot**: steal coins they don't have keys for; create new coins.

Cost to attack Bitcoin at 2026 hashrate: tens of millions of USD/hour. Cost to attack a small PoW altcoin: <$1k/hour (see crypto51.app).

3.8 Lightning Network (L2)

A payment channel network on top of Bitcoin.

```
1. Alice and Bob open a 2-of-2 multisig channel with 1 BTC each.
2. They update balances off-chain by exchanging signed commitment txs.
3. Either party can broadcast the latest commitment to close.
4. HTLCs allow routing payments across multi-hop channels.
```

Throughput: theoretically millions of TPS. Tradeoffs: liquidity locked in channels, online requirements (watchtowers), routing UX problems.

3.9 SegWit, Taproot, and the upgrade path

Upgrade	Year	What it did
SegWit (BIP-141)	2017	Separated signature data → fixed malleability, enabled Lightning
Taproot (BIP-340/341/342)	2021	Schnorr signatures + Tapscript → privacy + multi-sig efficiency
Inscriptions / Ordinals	2023	Unintended use of Taproot data → on-chain NFTs
Future: covenants (OP_CAT, CTV)	TBD	Programmable spending conditions, vaults

---

3.10 What Bitcoin is *not*

- Not anonymous — pseudonymous; chain analysis routinely de-anonymizes addresses.
- Not free — fees fluctuate $0.50 to $50+.
- Not energy-efficient — ~150 TWh/yr, comparable to a mid-size country.
- Not fast — 7 tx/sec, 60-minute finality.

These limitations created the design space every other blockchain explores.

3.11 Reading list

- Nakamoto — *Bitcoin: A Peer-to-Peer Electronic Cash System*, 2008.
- Eyal & Sirer — *Majority is not Enough: Bitcoin Mining is Vulnerable*, FC 2014.
- Antonopoulos — *Mastering Bitcoin* (2nd ed).
- Garay, Kiayias, Leonardos — *The Bitcoin Backbone Protocol*, Eurocrypt 2015.
- Sompolinsky & Zohar — *Secure High-Rate Transaction Processing in Bitcoin*, FC 2015.
- Poon & Dryja — *Lightning Network paper*, 2016.

3.12 Lab (3 hrs)

1. Connect to a Bitcoin testnet node (use `bitcoind` regtest or a public Esplora API).
2. Generate two keypairs, fund one from a faucet, create + sign + broadcast a transaction to the second.
3. Decode the resulting transaction and identify every field.
4. Inspect 3 real mainnet blocks and compute their fee revenue vs subsidy.

3.13 Quiz

1. Why does Bitcoin use the UTXO model instead of account balances?
2. What probability of reversal does Nakamoto's analysis give for `q=0.3, z=6`?
3. Explain selfish mining in one sentence and the minimum hashrate for it to be profitable.
4. What is the maximum theoretical TPS on Bitcoin L1, and what limits it?
5. Name two changes Taproot enabled and one second-order effect (e.g., Ordinals).

Blockchain Module 4 — Ethereum & Smart Contracts

> ***Goal**: Understand Ethereum as a programmable state machine and the design choices that opened (and complicated) the smart-contract ecosystem.*

4.1 The Ethereum thesis

Vitalik Buterin (2013): generalize Bitcoin's Script into a **Turing-complete VM** so that arbitrary financial logic — not just payments — can be enforced on-chain. The result was the EVM (Ethereum Virtual Machine).

Account model vs UTXO

- **Externally Owned Accounts (EOAs)** — controlled by a private key.
- **Contract Accounts** — controlled by code (deployed at an address).

Each account has: `nonce, balance, storage_root, code_hash`. State is a Merkle Patricia Trie of all accounts.

4.2 The EVM

A 256-bit, stack-based virtual machine.

Component	Purpose
Stack	1024 slots, 256-bit words
Memory	Volatile, byte-addressable
Storage	Persistent, key→value, per contract
Gas	Metering — every opcode costs gas
Logs	Off-chain indexable events (used by The Graph)

Gas

Every operation consumes gas. Transaction sender sets `gas_limit` and `gas_price`. If gas runs out, execution reverts but fees are still paid. This is how the EVM avoids the halting problem despite Turing completeness.

Op	Gas cost
ADD	3
SLOAD (read storage)	2100 (cold) / 100 (warm)
SSTORE (write storage)	20,000 (new) / 5,000 (modify)
CALL	700 + memory

These costs incentivize gas-aware contract design — a discipline as deep as low-latency C.

4.3 Solidity — the canonical language

```
pragma solidity ^0.8.20;

contract ERC20 {
    mapping(address => uint256) public balanceOf;
    uint256 public totalSupply;

    event Transfer(address indexed from, address indexed to, uint256 value);

    function transfer(address to, uint256 amount) external returns (bool) {
        require(balanceOf[msg.sender] >= amount, "insufficient");
        balanceOf[msg.sender] -= amount;
        balanceOf[to]         += amount;
        emit Transfer(msg.sender, to, amount);
        return true;
    }
}
```

Other languages: **Vyper** (Python-like, safer subset), **Huff** (low-level for gas-golf), **Fe** (Rust-inspired).

4.4 ERC standards

ERC	Purpose	Where it shows up
ERC-20	Fungible tokens	USDC, DAI, WETH
ERC-721	Non-fungible tokens (NFTs)	CryptoPunks, BAYC
ERC-1155	Multi-token (fungible + NFT)	Game items, OpenSea
ERC-4626	Tokenized vaults	DeFi yield aggregators
ERC-4337	Account abstraction	Smart-contract wallets
ERC-6551	Token-bound accounts	NFTs that *are* wallets

Each standard is a Schelling point — a contract that conforms slots seamlessly into the rest of the ecosystem.

4.5 The famous reentrancy bug (and the DAO hack)

```
// VULNERABLE
function withdraw() external {
    uint256 bal = balances[msg.sender];
    (bool ok,) = msg.sender.call{value: bal}("");   // ← attacker re-enters here
    require(ok);
    balances[msg.sender] = 0;                       // ← updated AFTER the call
}
```

If `msg.sender` is a contract, its `receive()` function runs *during* the external call, and it can call `withdraw()` again

before `balances[msg.sender]` is zeroed.

The 2016 DAO hack drained 3.6M ETH this way (~$60M then; $billions today). The fix:

```
// CHECKS-EFFECTS-INTERACTIONS pattern
function withdraw() external {
    uint256 bal = balances[msg.sender];
    balances[msg.sender] = 0;                          // effect FIRST
    (bool ok,) = msg.sender.call{value: bal}("");    // interaction LAST
    require(ok);
}
```

The DAO hack triggered the contentious Ethereum / Ethereum Classic hard fork — a foundational lesson in protocol governance.

4.6 The Merge & PoS

September 15, 2022: Ethereum switched from PoW to PoS in a flawless coordination feat ("The Merge"). PoS reduced energy consumption by ~99.95%.

Validator deposit: 32 ETH. Slashing for double-signing or surround voting. Block proposers are selected by RANDAO + VRF.

Finality via **Casper FFG** — a checkpoint finality layer on top of the chain. Every epoch (32 slots × 12s = 6.4 min) validators vote on a checkpoint; once a checkpoint has 2/3 attestations *twice*, it's finalized.

4.7 Account abstraction (ERC-4337)

Traditional wallets are EOAs — keys = identity = trust root. ERC-4337 lets *any* contract act as a wallet via `UserOperations` going through a `Bundler` and `EntryPoint`. This unlocks:

- Social recovery (lose your keys, restore via friends)
- Session keys (sign once, transact for a session)
- Sponsored gas (someone else pays your gas)
- Multi-factor / passkey signing

This is the largest UX shift in Ethereum since deployment.

4.8 Layer 2 ecosystem (preview — full coverage in Module 6)

L2	Type	TPS
Arbitrum	Optimistic rollup	~40k
Optimism	Optimistic rollup	~30k
Base	Optimistic (Coinbase)	~30k
zkSync Era	ZK rollup	~100k*

StarkNet	ZK rollup (Cairo)	~100k*
Polygon zkEVM	ZK rollup	~50k*
Scroll	ZK rollup	~20k*

(* peak theoretical)

4.9 The MEV economy

Block proposers can reorder, insert, or censor transactions for profit (Maximal Extractable Value). MEV-Boost separates **block proposers** from **block builders** — proposers auction off the right to build a block. Roughly 90%+ of Ethereum validators run MEV-Boost.

This created **Proposer-Builder Separation (PBS)** as a research field — see Module 9.

4.10 Reading list

- Buterin — *Ethereum White Paper*, 2013.
- Wood — *Ethereum Yellow Paper*, 2014–present.
- Antonopoulos & Wood — *Mastering Ethereum*, O'Reilly.
- Daian et al. — *Flash Boys 2.0: Frontrunning, Transaction Reordering, and Consensus Instability in DEXes*, S&P 2020.
- Buterin & Griffith — *Casper the Friendly Finality Gadget*, 2017.
- ERC-4337 spec — eips.ethereum.org/EIPS/eip-4337.

4.11 Lab (3 hrs)

1. Install Foundry. `forge init` a project.
2. Write an ERC-20 with `mint`, `transfer`, and `burn`. Write tests achieving 100% line coverage.
3. Write a deliberately reentrancy-vulnerable bank contract and an attacker contract that drains it.
4. Apply the checks-effects-interactions fix and prove the attack now fails.
5. Deploy to Sepolia testnet. Verify on Etherscan.

4.12 Quiz

1. State three differences between the UTXO and account models.
2. Why is gas necessary even though the EVM is Turing-complete?
3. What is the checks-effects-interactions pattern and which class of bugs does it prevent?
4. Explain the role of RANDAO and VRF in PoS proposer selection.
5. What problem does ERC-4337 solve, and at what cost?

Unit 3

Consensus Algorithms

Required Contact Hours: **7**

- Proof-of-Work variants and tradeoffs
- Proof-of-Stake: Casper FFG, Tendermint, Ouroboros
- PBFT, HotStuff, pipelined 3-phase voting
- Algorand BA*, Avalanche Snowball, Solana PoH
- Slashing, nothing-at-stake, long-range attacks

The chapters that follow expand each topic above with theory, worked examples, reading lists, hands-on lab guidance, and end-of-chapter self-assessment questions.

Blockchain Module 5 — Consensus Algorithms

> ***Goal**: Compare every major consensus mechanism on the same axes: security model, finality type, throughput, decentralization, and known attacks.*

5.1 The consensus design space

A consensus protocol must answer:

1. **Who proposes** the next block? (Sybil-resistance mechanism)
2. **Who votes** on its validity?
3. **When is it final**?
4. **How are honest behavior and dishonest behavior incentivized / punished**?

Combinations of answers give us all known protocols.

5.2 Proof-of-Work (PoW)

Sybil resistance via computational cost.

Property	Value
Proposer selection	Solve hash puzzle
Finality	Probabilistic
Energy use	Very high
Hardware barrier	High (ASICs)
Throughput	Low
Known attacks	Selfish mining, 51%, eclipse

Variants:

- **SHA-256** (Bitcoin) — ASIC-friendly
- **Ethash** (pre-Merge Ethereum) — memory-hard, GPU-friendly (intentionally ASIC-resistant; partially failed)
- **RandomX** (Monero) — CPU-friendly via virtual machine
- **Scrypt, X11, Equihash** — earlier altcoin attempts

5.3 Proof-of-Stake (PoS)

Sybil resistance via locked economic value.

A validator deposits collateral; misbehavior is **slashed** (collateral burned). Honest behavior earns issuance + fee rewards.

Variants

Protocol	Mechanism	Used by
Casper FFG + LMD-GHOST	Hybrid finality + fork choice	Ethereum
Tendermint / CometBFT	Round-based BFT, instant finality	Cosmos chains
Ouroboros	Slot leader from VRF	Cardano
Algorand BA*	Cryptographic sortition + BA	Algorand
Snowball / Avalanche	Repeated random sampling	Avalanche
PoH + ToS	Verifiable delay + TowerBFT	Solana

Slashing conditions (Ethereum)

1. **Double proposal** — sign two different blocks for the same slot.
2. **Surround vote** — attesting to a checkpoint that surrounds your previous one.
3. **Inactivity leak** — if chain can't finalize, validators that aren't voting bleed stake.

"Nothing at stake" (early PoS problem, solved)

In naive PoS, a validator could vote on every fork (it's costless), preventing convergence. Slashing solves this by making conflicting votes punishable.

5.4 Proof-of-Authority (PoA)

A fixed set of permissioned validators. Used for consortium chains (POA Network, Binance Smart Chain, many enterprise blockchains).

Pros: very high throughput. Cons: trust-based, single-org risk.

5.5 Practical Byzantine Fault Tolerance (PBFT, 1999)

```
1. Pre-prepare:  leader proposes
2. Prepare:      all replicas exchange "I saw the proposal"
3. Commit:       all replicas exchange "I'm ready to commit"
4. Reply:        once 2f+1 commits, finalized
```

$O(n^2)$ messages per round → doesn't scale beyond ~100 nodes. HotStuff (2019) reduced this to linear with chained pipelined voting.

5.6 HotStuff

The basis of Diem, Aptos, and modern BFT chains.

- **Linear view change** — replacing a faulty leader costs O(n) messages instead of O(n²).
- **Pipelined** — three-phase QC chained so each block contributes to multiple commits.
- **Optimistic responsiveness** — protocol moves at network speed when network is healthy.

5.7 Tendermint / CometBFT

Used by Cosmos chains.

- Round-based: propose → pre-vote → pre-commit → commit.
- Instant absolute finality (1 block).
- Validator set fixed per epoch.
- Cannot fork — if 2/3 are honest, only one block per height.
- Limit: ~100–200 validators (gossip cost).

5.8 Ouroboros (Cardano)

Time is divided into **epochs** → **slots**. A VRF selects a slot leader. Multiple variants:

- **Ouroboros Classic** — synchronous, multi-party coin tossing.
- **Ouroboros Praos** — semi-synchronous, secure under adaptive corruption.
- **Ouroboros Genesis** — full bootstrap security (no trusted setup of recent chain).
- **Ouroboros Hydra** — L2 with off-chain state channels.

Notable for being one of the most academically rigorous protocol families.

5.9 Algorand BA*

Pure PoS with cryptographic sortition.

- Each round, ~1000 validators are sampled (private VRF reveal).
- BA* algorithm runs Byzantine agreement in ~5 seconds.
- No forks, instant finality.
- Designed under partial synchrony with adaptive corruption.

5.10 Avalanche consensus (Snowball/Snowflake/Avalanche)

A **gossip-and-sample** family:

```
loop:
  query K random peers for their preference
  if α·K agree, increment confidence
  if confidence > β, decide
```

Sub-second finality, very high throughput. Used by Avalanche network.

5.11 Proof-of-History (Solana)

A VDF (verifiable delay function) produces a cryptographic clock so validators don't need to negotiate timestamps. Combined with TowerBFT for finality.

Pros: massive throughput claims (50k+ TPS). Cons: high hardware requirements → centralization pressure; several major outages.

5.12 Comparison table

Protocol	Finality	TPS (real)	Validators	Energy	Sybil
Bitcoin PoW	Probabilistic 60min	7	~thousands (miners)	~150 TWh/yr	Compute
Ethereum PoS	12.8min absolute	15 (L1), 100k+ (L2)	~1M	Low	Stake
Cosmos (CometBFT)	1 block, ~6s	~1k	~150	Low	Stake
Solana (PoH+TowerBFT)	~13s	4–6k typical	~2k	Mid	Stake
Avalanche	<2s	1–5k	~1.5k	Low	Stake
Cardano (Ouroboros)	Probabilistic	~250	~3k stake pools	Low	Stake

---

5.13 Reading list

- Castro & Liskov — *Practical Byzantine Fault Tolerance*, 1999.
- Yin et al. — *HotStuff: BFT Consensus with Linearity and Responsiveness*, PODC 2019.
- Kiayias et al. — *Ouroboros: A Provably Secure Proof-of-Stake Blockchain Protocol*, CRYPTO 2017.
- Chen & Micali — *Algorand: A Secure and Efficient Distributed Ledger*, 2017.
- Rocket et al. — *Scalable and Probabilistic Leaderless BFT Consensus through Metastability* (Avalanche).
- Yakovenko — *Solana: A new architecture for a high performance blockchain*, 2018.
- Buterin & Griffith — *Casper the Friendly Finality Gadget*, 2017.

5.14 Lab (3 hrs)

Pick **two** protocols and implement a stripped-down simulator:

1. Tendermint round (propose → prevote → precommit → commit) with 4 nodes, 1 Byzantine.
2. Avalanche Snowball with 100 nodes — show convergence as α and K vary. Plot finality time vs Byzantine fraction.

5.15 Quiz

1. Why does naive PoS suffer from "nothing at stake" and how does slashing fix it?

2. State HotStuff's improvement over PBFT in big-O terms.
3. What's the role of the VRF in Ouroboros and Algorand?
4. Explain why Solana's PoH is *not* a consensus mechanism by itself.
5. Why is Tendermint capped at ~150 validators in practice?

Unit 4

Scalability, Layer-2 and Zero-Knowledge Proofs

Required Contact Hours: **9**

- Scalability Trilemma, sharding, vertical vs horizontal scaling
- Optimistic Rollups — fraud proof bisection (Arbitrum, Optimism)
- ZK Rollups — zkSync, StarkNet, Polygon zkEVM
- Data availability — EIP-4844 blobs, Celestia, EigenDA
- ZKPs — SNARK vs STARK, Groth16, PLONK, Circom toolchain

The chapters that follow expand each topic above with theory, worked examples, reading lists, hands-on lab guidance, and end-of-chapter self-assessment questions.

Blockchain Module 6 — Scalability & Layer 2

> ***Goal**: Internalize the **scalability trilemma**, understand every major L2 design (rollups, channels, sidechains, validiums, plasma), and reason about which design fits which application.*

6.1 The scalability trilemma (Vitalik, 2017)

> *A blockchain can have at most two of: **Scalability, Security, Decentralization.***

Strictly informal but useful as a planning lens:

- Bitcoin/Ethereum L1: Security + Decentralization, weak Scalability.
- BSC / EOS: Scalability + Security, weak Decentralization.
- Many sidechains: Scalability + Decentralization, weaker Security.

L2s aim to inherit L1 Security while gaining Scalability.

6.2 Scaling axes

Axis	Lever
Vertical	Bigger blocks, faster nodes (Solana)
Horizontal	Sharding (Near, Ethereum data sharding)
Off-chain	Channels (Lightning)
Rollups	Bundle txs off-chain, post compressed proof on-chain
Sidechains	Independent chain, bridged

6.3 State / payment channels

```
Open:    on-chain multisig deposit
Update:  off-chain signed state transitions
Close:   on-chain settlement
```

- **Bitcoin Lightning** — payment channels with HTLC routing.
- **Raiden** — Lightning for Ethereum ERC-20s.
- **Connext, Hydra (Cardano)** — generalized state channels.

Pros: instant, near-free. Cons: requires liquidity, online watchtowers, doesn't generalize beyond fixed participants.

6.4 Plasma (deprecated but instructive)

Vitalik & Joseph Poon, 2017. A child chain commits Merkle roots to the parent chain. Users could exit fraudulent state.

Why it died: exit games are hard, mass exit problem (one operator hides data → all users must exit within window), unsuited to general smart contracts.

The exit-game intellectual heritage lives on in **Optimistic Rollups**.

6.5 Sidechains

A separate blockchain with its own consensus, bridged to L1.

Example	Mechanism
Polygon PoS	Heimdall consensus + checkpoint commits to Ethereum
Gnosis Chain	Independent PoS, xDAI
Liquid (Bitcoin)	Federation-secured

Security is **independent of L1** — you only trust the sidechain's own consensus. A bridge hack is a sidechain hack.

6.6 Rollups — the dominant L2 paradigm

A **rollup** executes transactions off-chain, posts:

- **Compressed transaction data** to L1 (data availability)
- **A validity argument** that the new state is correct

Two flavors

Type	Validity argument
Optimistic Rollup	Assume correct; allow fraud proofs within a 7-day challenge window
ZK Rollup (a.k.a. Validity Rollup)	Post a zk-SNARK / zk-STARK proving correctness — instant finality

Why both exist

	Optimistic	ZK
Finality	7 days	minutes
Gas cost on L1	Cheap	Currently expensive (prover cost) but shrinking
EVM equivalence	Easy	Hard (zkEVM is bleeding edge)
Maturity	Higher	Catching up fast

6.7 Optimistic Rollups in depth

Examples: **Arbitrum One, Optimism, Base, Mantle, Blast**.

Fraud proof game (interactive):

1. Sequencer posts a state root.
2. Within 7 days, anyone can challenge.
3. Bisection narrows down to the disputed step.
4. L1 EVM executes that single step. Whoever was wrong loses bond.

This is the "bisection" trick that makes fraud proofs O(log n) instead of O(n).

Arbitrum vs Optimism

- **Arbitrum** uses a WASM-based interactive proof and has a custom precompile model.
- **Optimism** built **OP Stack**, a modular L2 framework now powering Base, Worldcoin, Zora, etc. → "Superchain" vision.

6.8 ZK Rollups in depth

Examples: **zkSync Era, StarkNet, Polygon zkEVM, Scroll, Linea**.

The sequencer:

1. Executes a batch of txs.
2. Generates a SNARK/STARK proving `(prev_state, txs) → new_state` is valid under EVM semantics.
3. Posts proof + minimal data to L1.

L1 verifies the proof (~200k gas). Done.

zkEVM equivalence levels (Vitalik's classification)

- **Type 1**: byte-exact EVM (Taiko aims here)
- **Type 2**: equivalent for any Solidity program (Scroll, Polygon zkEVM)
- **Type 2.5**: minor gas-cost differences
- **Type 3**: most contracts work (early zkSync)
- **Type 4**: high-level language compiles to ZK-native (StarkNet/Cairo, zkSync's Era)

Type 1 is hardest (most EVM-compatible, slowest proving). Type 4 is easiest (custom VM).

6.9 Data availability (DA) — the next bottleneck

Rollups *must* post tx data on L1 so anyone can reconstruct state. Posting raw data is expensive.

Solutions

Solution	Approach
EIP-4844 (proto-danksharding)	Blob transactions — cheap, ephemeral (18 days), 128KB blobs

Full Danksharding	128 blobs per slot, ~1.3MB/slot DA → ~100k TPS L2
Celestia	Standalone DA layer using data availability sampling (DAS)
EigenDA	Restaking-secured DA on EigenLayer
Avail	Polygon's standalone DA chain
Validium	DA off-chain (DAC committee) — cheaper but weaker security

DAS lets light clients verify with high probability that data was published, without downloading all of it — see Mustafa Al-Bassam's thesis.

6.10 Hybrid: Volitions and DACs

A **volition** lets users choose per-transaction whether their data goes on L1 (rollup security) or off-chain (validium speed). StarkEx pioneered this.

6.11 The "modular blockchain" thesis

Decompose monolithic blockchains into:

Layer	Responsibility	Examples
Execution	Run tx state transitions	rollups, OP Stack, Arbitrum Orbit
Settlement	Resolve disputes, anchor canonicity	Ethereum
Data availability	Make tx data retrievable	EigenDA, Celestia, Avail, blobs
Consensus	Order blocks	Ethereum, Tendermint

Cosmos, Celestia, EigenLayer, and the OP Superchain are all bets on modularity. **Solana** is the monolithic counter-bet.

6.12 Shared sequencers & cross-rollup atomicity

If every rollup has its own sequencer, cross-rollup composability dies. Active research:

- **Espresso, Astria** — shared sequencer networks
- **SUAVE (Flashbots)** — decentralized block builder
- **Booster Rollups** — designed to share execution

6.13 Reading list

- Buterin — *An Incomplete Guide to Rollups*, 2021 (vitalik.eth.limo).
- Al-Bassam, Sonnino, Buterin — *Fraud and Data Availability Proofs*, 2018.
- Kalodner et al. — *Arbitrum: Scalable, private smart contracts*, USENIX Security 2018.
- Ben-Sasson et al. — *Zerocash* + *zk-STARK* papers.

- *EIP-4844: Proto-Danksharding*.
- *Celestia whitepaper*.
- Buterin — *Endgame*, 2021.

6.14 Lab (3 hrs)

1. Deploy the same Solidity contract on Sepolia (L1) and Optimism Sepolia (L2). Compare gas cost.
2. Use `op-batcher` data on Etherscan to inspect a real L2 batch.
3. Read a single blob from a Beacon node via `/eth/v1/beacon/blob_sidecars/{slot}`.
4. Compute the cost-per-byte savings of EIP-4844 vs calldata.

6.15 Quiz

1. State the scalability trilemma and one counter-example to it.
2. Why do Optimistic Rollups need a 7-day challenge window?
3. What's the difference between a Type-2 and Type-4 zkEVM?
4. Define data availability sampling in one sentence.
5. What does EIP-4844 reduce, and by roughly how much?

Blockchain Module 7 — Privacy & Zero-Knowledge Proofs

> ***Goal**: Build the mental model for ZK proofs — what they prove, what they hide, and the algebraic machinery that makes it possible. ZK is the most transformative cryptographic tool of the decade.*

7.1 What is a zero-knowledge proof?

A protocol by which a **prover** convinces a **verifier** that a statement $x \in L$ (some language L) is true, without revealing anything beyond that fact.

Three properties

1. **Completeness** — if $x \in L$, an honest prover convinces an honest verifier.
2. **Soundness** — if $x \notin L$, no malicious prover can convince an honest verifier (except with negligible probability).
3. **Zero-knowledge** — the verifier learns nothing about the witness w beyond $x \in L$.

Formalized by Goldwasser, Micali, Rackoff, 1985 (STOC).

7.2 Why ZK matters for blockchains

Use case	Why ZK
Privacy (Zcash, Aleo)	Hide sender, receiver, amount
Scalability (ZK rollups)	One proof verifies thousands of txs
Identity (Worldcoin, Sismo)	Prove "I am unique human" without revealing who
Compliance (Aztec, zkKYC)	Prove "I am not on a sanctions list" without revealing identity
ML (zkML)	Prove inference was done correctly without revealing model or input

---

7.3 Interactive vs non-interactive

Type	Form	Use case
Interactive (IP)	Multi-round dialogue	Theory, identification protocols
Non-interactive (NIZK)	Single message via random oracle (Fiat-Shamir)	Blockchains (no interaction with miners possible)

Fiat-Shamir heuristic: replace verifier challenges with hash of transcript. Compatible with public broadcast.

7.4 The SNARK / STARK family

Property	SNARK	STARK
Trusted setup	Often needed (Groth16)	Never
Proof size	~200 bytes	~50–200 KB
Verifier time	~5 ms	~10–40 ms
Prover time	Slow	Faster (with FRI)
Post-quantum	No (most)	Yes
Crypto basis	Pairings (BN254/BLS12-381)	Hash functions only

SNARK construction families

- **Groth16** (2016) — smallest proofs, per-circuit trusted setup. Used by Zcash Sapling.
- **PLONK** (2019) — universal trusted setup (one per chain). Halo2, plonky2/3 build on this.
- **Marlin** — universal setup, smaller proofs than PLONK in some settings.
- **Halo / Halo2** — no trusted setup, recursive composition. Used by Zcash Orchard, Mina.
- **Nova / SuperNova / HyperNova** — incrementally verifiable computation (IVC) via folding. Hot 2023–2026 research.

STARK

Ben-Sasson, Bentov, Horesh, Riabzev (2018). Transparent (no trusted setup), post-quantum, uses FRI for polynomial commitments. Used by StarkNet.

7.5 The circuit abstraction

To prove "I know `w` such that `C(x, w) = 0`", we express `C` as an **arithmetic circuit** over a finite field.

```
Inputs: public x, private w
Each gate: addition or multiplication over F_p
Output: 0 if predicate holds
```

Then we reduce circuit satisfaction to a polynomial identity (e.g., **R1CS → QAP** for Groth16, **AIR → FRI** for STARKs).

7.6 A concrete tiny example

Prove "I know `x` such that `H(x) = y`" without revealing `x`:

1. Write SHA-256 as an arithmetic circuit (~25k constraints in R1CS).
2. Run the SNARK prover with witness `x` and public input `y`.
3. Verifier checks the proof.

This is what every privacy coin does for `(sender, amount)`.

7.7 zkSNARKs in writing real circuits

Modern toolchains:

DSL / framework	Backend	Used for
Circom	Groth16, PLONK	Hands-on, large ecosystem
Noir (Aztec)	UltraPlonk	Rust-like syntax
Cairo (StarkWare)	STARK	StarkNet
Halo2	Halo2 / IPA	zkEVMs (Scroll)
Risc Zero zkVM	STARK + recursion	"Prove any Rust program"
SP1 (Succinct)	STARK / Plonky3	General zkVM

zkVMs (Risc Zero, SP1, Jolt, Zeth) are the "compiler" wave — instead of hand-writing circuits, compile arbitrary programs into ZK-provable form.

7.8 Privacy-preserving cryptocurrencies

Project	Mechanism
Zcash	zk-SNARK over Sapling/Orchard circuits
Monero	Ring signatures + RingCT + stealth addresses (not ZK, but related)
Aleo	zkSNARK-based programmable privacy
Aztec	zk-private smart contracts (Noir + UltraPlonk)
Penumbra	Private DeFi in the Cosmos ecosystem

7.9 Multi-Party Computation (MPC) and FHE (briefly)

Tool	What it does
MPC	Multiple parties compute `f(x1,...,xn)` without revealing individual `xi`
FHE	Compute on encrypted data; only key holder can decrypt result
TEE	Hardware enclave (Intel SGX, AMD SEV) for confidential computation

These are complements to ZK, not substitutes. ZK proves correctness; MPC/FHE compute confidentially.

7.10 ZK + AI = zkML (preview)

Active research: prove a neural network's inference was performed correctly without revealing the model weights (or the input).

Why hard:

- Floating-point → field arithmetic
- Non-linear ops (ReLU, softmax) → expensive constraints
- Large models → billions of constraints

Frameworks: **EZKL, ZKonduit, Modulus Labs, Giza, Risc Zero zkML**.

Covered in Intersection module.

7.11 Reading list

- Goldwasser, Micali, Rackoff — *The knowledge complexity of interactive proof systems*, 1985.
- Groth — *On the size of pairing-based non-interactive arguments*, 2016.
- Gabizon, Williamson, Ciobotaru — *PLONK*, 2019.
- Ben-Sasson et al. — *Scalable, transparent, and post-quantum secure computational integrity*, 2018.
- Bowe, Grigg, Hopwood — *Halo: Recursive proof composition without a trusted setup*, 2019.
- Kothapalli, Setty, Tzialla — *Nova*, CRYPTO 2022.
- Boneh, Drake — *zkSNARKs in practice* (Stanford lectures).

7.12 Lab (4 hrs)

1. Install Circom + snarkjs.
2. Write a circuit that proves knowledge of `x` such that `x*x = 25` without revealing whether `x=5` or `x=-5`.
3. Compile, do trusted setup, generate proof, verify on-chain (deploy verifier contract to Sepolia).
4. Then write a Poseidon hash circuit and a Merkle inclusion proof. (Use existing libs.)

7.13 Quiz

1. State the three properties of a ZKP.
2. What problem does Fiat-Shamir solve?
3. Compare Groth16 and PLONK on (setup, proof size, prover time).
4. Why are STARKs considered post-quantum and SNARKs (mostly) not?
5. What is a zkVM and how does it differ from writing a custom circuit?

Unit 5

DeFi, Security, Governance and Research Frontiers

Required Contact Hours: **12**

- DeFi — Stablecoins, AMMs, lending markets, oracles
- Flash loans and MEV (sandwich, arbitrage, PBS)
- Smart contract security — Reentrancy, access control, postmortems
- Governance and tokenomics — DAOs, veTokens, EigenLayer
- Advanced consensus — DAGs, HotStuff lineage, restaking
- Research frontiers — zkML, zkVMs, folding schemes, modular blockchains

The chapters that follow expand each topic above with theory, worked examples, reading lists, hands-on lab guidance, and end-of-chapter self-assessment questions.

Blockchain Module 8 — DeFi Architecture

> ***Goal**: Understand DeFi as a system of **composable financial primitives**, the math behind AMMs, and the systemic risks (oracles, MEV, governance attacks).*

8.1 What makes DeFi different

DeFi = **Decentralized Finance** — financial applications running on permissionless smart contract platforms. Distinctive properties:

- **Non-custodial** — users hold their own keys.
- **Composable ("money legos")** — protocols call other protocols permissionlessly.
- **Public state** — every position is visible and analyzable.
- **24/7** and global.

These are also its risk surface.

8.2 The DeFi stack

Layer	Examples
Stablecoins	USDC, USDT, DAI, FRAX, GHO
DEXes	Uniswap, Curve, Balancer, dYdX
Lending	Aave, Compound, Morpho, Spark
Derivatives	dYdX, GMX, Synthetix, Hyperliquid
Yield aggregators	Yearn, Beefy, Pendle
Liquid staking	Lido, Rocket Pool, EtherFi
Restaking	EigenLayer, Symbiotic, Karak
Bridges	Wormhole, LayerZero, Across, Hop
Oracles	Chainlink, Pyth, RedStone, UMA

8.3 Automated Market Makers (AMMs)

Replace order books with a deterministic price function `f(x, y) = k`.

Uniswap v2 — Constant Product

```
x * y = k
```

If you swap `dx` X-tokens in, you receive `dy = y - k/(x+dx)` Y-tokens. The price slips along the curve.

Slippage and impermanent loss

- **Slippage** — large trades move price along curve.
- **Impermanent loss (IL)** — LPs lose vs. just holding when prices diverge:

```
IL(r) = 2·sqrt(r)/(1+r) - 1,  where r = price_now/price_initial
```

A 2× price move → ~5.7% IL. A 5× move → ~25%.

Curve — StableSwap

Optimized for assets that trade near parity (USDC/DAI). Hybrid of constant-sum (near parity) and constant-product (at extremes), via an amplification coefficient `A`.

Uniswap v3 — Concentrated Liquidity

LPs supply liquidity within a **price range** `[p_a, p_b]`. Capital efficiency 100×–4000× higher than v2, but requires active management and IL is amplified.

Balancer

Weighted constant-product across N tokens:

```
∏ x_i^w_i = k,  where Σ w_i = 1
```

Lets you build custom index funds as AMMs.

Uniswap v4 — Hooks

Pools become extensible — anyone can attach pre/post hooks (custom fees, dynamic fees, on-chain limit orders). The standard for AMM customization in 2024–2026.

8.4 Lending markets

Two dominant designs:

Pool-based (Aave, Compound)

Suppliers deposit into a pool, borrowers draw with overcollateralization. Interest rate is a deterministic function of utilization.

```
borrow_rate = base + slope1·U + slope2·max(U - kink, 0)
```

Isolated / Peer-to-Pool (Morpho)

Match P2P when possible, fall back to pool. Improves capital efficiency.

Liquidation

If collateral / debt < `liquidation_threshold`, anyone can repay part of debt in exchange for collateral at a discount (liquidation bonus). This keeps the system solvent.

Liquidation cascades during 2020's "Black Thursday" and 2022's collapse showed the system works, but with brutal slippage.

8.5 Stablecoins

Type	Mechanism	Example
Fiat-backed	1:1 USD reserves	USDC, USDT
Crypto-backed (CDP)	Over-collateralized vaults	DAI, LUSD
Algorithmic	Mint/burn against another token	UST (failed catastrophically), AMPL
Hybrid / partially-collateralized	Mix	FRAX (legacy), GHO
LSD-backed	Liquid-staking tokens as collateral	eUSD, USDe

The **Terra/UST collapse** (May 2022) eliminated the pure-algo design space for years.

8.6 Oracles — DeFi's biggest single risk

Smart contracts can't read external prices. They depend on **oracles**.

Type	Example
Push	Chainlink price feeds update on-chain at threshold/interval
Pull	Pyth — signed prices fetched on demand
Optimistic	UMA — assertions, dispute window
TWAP	Uniswap v3 time-weighted price

Oracle attacks

Flash-loan-fueled price manipulation drained Mango Markets ($117M, 2022), bZx, Cream, and many others. Mitigations:

- Use TWAPs (manipulating averages requires sustained capital).
- Use chain-of-custody-signed oracles (Pyth Pull).
- Cross-check multiple sources.

8.7 Flash loans

Borrow any amount with **no collateral**, provided the loan is repaid in the same transaction. If not, the tx reverts

atomically.

```
function flashLoan(uint256 amount, bytes calldata data) external {
    uint256 before = token.balanceOf(address(this));
    token.transfer(msg.sender, amount);
    IFlashBorrower(msg.sender).onFlashLoan(amount, data);
    require(token.balanceOf(address(this)) >= before + fee, "not repaid");
}
```

Legitimate use: arbitrage, collateral swaps, self-liquidation. Attack use: capital-free price manipulation (Mango, bZx, Cream).

The asymmetry — anyone can borrow $100M for one transaction — is uniquely DeFi.

8.8 MEV (Maximal Extractable Value)

Profit available to whoever orders transactions. Categories:

Category	Example
Arbitrage	Same asset different prices on two DEXes
Sandwich	Insert buy before / sell after a victim's swap
Liquidation	Race to liquidate underwater positions
Long-tail	NFT mints, contract-state predictions

Flashbots introduced auctions for MEV (MEV-Boost). On Ethereum, ~90%+ of validators run MEV-Boost, separating proposers from builders (**PBS**).

MEV mitigation

- **Encrypted mempools** (Shutter, SUAVE)
- **Order flow auctions** (CoWSwap, UniswapX)
- **Fair ordering** (Themis, Pompê)
- **Threshold encryption** (Osmosis)

8.9 Composability — and its dark side

Aave → Lido → Curve → Yearn → Convex — a single user position can span 5+ protocols. This is brilliant and dangerous:

- **Contagion** — one exploit propagates through every composing protocol.
- **Reentrancy across protocols** — calls back through hooks can violate invariants.
- **Read-only reentrancy** — even view functions are unsafe during external calls.

Major incidents: Curve reentrancy (2023, $73M), Euler ($197M), Compound oracle glitch.

8.10 Restaking and shared security

EigenLayer (2023): let ETH stakers opt-in to securing additional services (oracles, DA layers, sidechains, AVSs). Liquid restaking tokens (LRTs) compound positions.

Risk: **systemic slashing** — a single AVS bug could cascade. Aggregate restaked ETH at peak: tens of billions of USD.

8.11 Reading list

- Adams, Zinsmeister, Salem, Keefer, Robinson — *Uniswap v3 Core*, 2021.
- Egorov — *StableSwap — efficient mechanism for Stablecoin liquidity*, 2019.
- Daian et al. — *Flash Boys 2.0*, S&P 2020.
- Qin, Zhou, Gervais — *Quantifying Blockchain Extractable Value*, S&P 2022.
- Werner et al. — *SoK: Decentralized Finance (DeFi)*, AFT 2022.
- Lo, Verma — *DeFi protocols for loanable funds*, AFT 2021.
- Klages-Mundt, Minca — *(In)stability for the Blockchain*, 2020 (algorithmic stablecoin analysis).

8.12 Lab (3 hrs)

1. Fork Ethereum mainnet locally with `anvil --fork-url`.
2. Deposit ETH into Uniswap v3 ETH/USDC pool at a chosen tick range. Measure fees earned for 1 hour of simulated swaps.
3. Implement a flash-loan arbitrage between Uniswap v2 and Sushiswap (clone Uni v2). Find a path where it's profitable in fork state.
4. Write a postmortem of one major 2022–2024 DeFi exploit.

8.13 Quiz

1. Derive the IL formula `2√r/(1+r) − 1` for constant-product AMMs.
2. Why are TWAP oracles harder to manipulate than spot oracles?
3. Explain the difference between push and pull oracles.
4. State three legitimate uses of flash loans.
5. Why does Uniswap v3 concentrated liquidity require active management?

Blockchain Module 9 — Security & Attack Surface

> ***Goal**: Build a catalog of every major attack class. Read at least 3 postmortems. Internalize that most blockchain "hacks" are app-layer bugs, not consensus breaks.*

9.1 Threat models

Adversary	Capability
External attacker	Can call any public function, see all state
Malicious validator	Can reorder/censor (within their slots), double-sign (gets slashed)
Compromised oracle	Can push arbitrary prices
Compromised bridge	Can mint unbacked tokens on destination chain
Governance attacker	Can pass malicious proposal with enough tokens
Nation-state	Long-range attacks, mining cartel, regulatory capture

---

9.2 Smart-contract bugs

Top 10 classes

1. **Reentrancy** — external call before state update.
2. **Integer over/underflow** — pre-0.8 Solidity; still possible with `unchecked {}`.
3. **Access control** — missing `onlyOwner` modifiers, default-public functions.
4. **Front-running / MEV exposure** — public mempool reveals intent.
5. **Oracle manipulation** — single source of truth.
6. **Logic bugs** — math wrong, accounting wrong (most common in DeFi).
7. **Improper initialization** — unprotected `initialize()`.
8. **Delegatecall to untrusted code** — full storage access.
9. **`tx.origin` for auth** — phishable.
10. **Unbounded loops** — gas griefing, DoS.

Less obvious

- **Read-only reentrancy** — view functions can return stale values mid-call.
- **Cross-function reentrancy** — re-enter a *different* function that shares state.
- **Signature replay** — missing `nonce` or `chainId` lets one signature be reused.
- **Permit2 / approval phishing** — drainer scams.

9.3 Famous exploits — a starter list

Incident	Year	Loss	Class
The DAO	2016	$60M	Reentrancy → ETH/ETC fork
Parity multisig	2017	513k ETH frozen	Library kill
bZx	2020 (×2)	$1M	Oracle / flash loan
Poly Network	2021	$611M	Access control
Wormhole	2022	$325M	Signature verification
Ronin Bridge	2022	$625M	Validator key compromise
Beanstalk	2022	$182M	Flash-loan governance
Nomad	2022	$190M	Init bug, copy-paste pillage
Mango Markets	2022	$117M	Oracle manipulation
Euler Finance	2023	$197M	Donation-attack accounting
Curve Vyper compiler	2023	$73M	Compiler reentrancy lock bug
Multichain	2023	$130M	Insider / opaque
Atomic Wallet	2023	$100M+	Wallet-side compromise
Orbit Bridge	2024	$80M	Bridge key compromise

Lesson: Bridges and cross-chain are the single most-attacked surface.

9.4 Consensus-layer attacks

Attack	Target	Mechanism
51% attack	PoW chain	Majority hashrate rewrites history
Long-range attack	PoS chain	Old keys re-create alternate history (mitigated by weak subjectivity)
Eclipse attack	Single node	Isolate node from honest network → feed fake chain
Sybil attack	Network	Many fake identities; Sybil-resistance (PoW/PoS) defeats it
Selfish mining	PoW	Withhold blocks to gain disproportionate reward
Time-bandit	EVM (theoretical)	Reorg to capture historical MEV
Bribery	PoS validators	Pay validators to break protocol
Inactivity leak	PoS chains	If chain can't finalize, non-voting validators bleed stake

---

9.5 Bridges — the trillion-dollar honey pot

Bridges typically lock tokens on Chain A and mint wrapped tokens on Chain B. Trust models:

Model	Examples	Weakness
Multi-sig federation	early Wormhole, Ronin	Key compromise (Ronin: 5 of 9 keys)
Light client (IBC, etc.)	Cosmos IBC	Strong, but only works for compatible chains
ZK light client	zkBridge, Polyhedra	Strong, expensive
Optimistic	Across, Nomad	Long challenge window
LayerZero / Wormhole / Axelar	Relayer + Oracle separation	Trust assumptions on relayers/guardians

Vitalik's "trilemma for bridges": you can have 2 of {trust-minimization, generality, extensibility-across-chains}.

9.6 Frontend attacks

The chain may be secure but the **frontend** is HTML/JS served from CDNs.

- Curve's frontend was hijacked via DNS hijack (2022) → users signed malicious tx.
- BadgerDAO's CDN-hosted JS injected approval-drainer (2021, ~$120M).

Mitigations: IPFS-pinned + ENS-resolved frontends, hardware wallet "blind signing" warnings, wallet drainers protection (Blockaid, Wallet Guard).

9.7 Phishing and the user surface

Drainers ("Inferno Drainer", "Pink Drainer", "Angel Drainer") use:

- Fake mints/airdrops
- Permit / Permit2 signature theft
- SetApprovalForAll tricks
- Address poisoning

User-side losses to drainers exceeded $300M in 2023. The chain didn't fail — the UX did.

9.8 Formal verification & static analysis

Tool	What it does
Slither	Static analysis for Solidity
Mythril	Symbolic execution
Echidna	Property-based fuzzing
Foundry invariant tests	Invariant fuzzing
Certora Prover	SMT-backed formal verification
K-framework	Formal semantics for EVM (KEVM)

Halmos	Symbolic Foundry tests

Best practice 2026 stack: Foundry unit tests + invariant tests + Slither in CI + Certora spec for critical invariants + a professional audit before mainnet.

9.9 The audit industry

Top firms (alphabetical, partial): **Trail of Bits, OpenZeppelin, ConsenSys Diligence, Spearbit, Code4rena, Zellic, Cantina, Sherlock, Halborn**.

Audits do not prove correctness — they reduce risk. **Code4rena** competitive audits typically find more issues than fixed-fee audits but at higher cost.

9.10 Incentive-compatible disclosure

Practice	Detail
Bug bounty	Immunefi, HackerOne; payouts up to $10M for L1 critical
Whitehat retrievals	Some exploits returned for 10% bounty
War rooms	Coordinated multi-protocol response (e.g., Curve July 2023)
Pause / kill switches	Trade-off: safety vs decentralization purity

---

9.11 Reading list

- Atzei, Bartoletti, Cimoli — *A Survey of Attacks on Ethereum Smart Contracts*, POST 2017.
- Daian et al. — *Flash Boys 2.0*, S&P 2020.
- Heilman et al. — *Eclipse Attacks on Bitcoin's Peer-to-Peer Network*, USENIX Security 2015.
- Eyal & Sirer — *Majority Is Not Enough*, FC 2014.
- *SoK: Cross-Chain Communication* (any of several).
- Rekt News — postmortem corpus (rekt.news).

9.12 Lab (3 hrs)

1. Fork Curve's affected pool at the exact block before the July 2023 reentrancy. Reproduce the drain in Foundry.
2. Run Slither on a small Solidity project. Triage the findings.
3. Write an invariant test in Foundry that would have caught the bug.

9.13 Quiz

1. State three differences between reentrancy and read-only reentrancy.
2. Why are bridges disproportionately attacked?
3. Define long-range attack and the standard mitigation.
4. What problem does MEV-Boost solve, and what new one does it introduce?

5. Name one tool from each of: static analysis, fuzzing, formal verification.

Blockchain Module 10 — Governance & Tokenomics

> ***Goal**: Understand how decisions get made in protocols, why token economics shapes (and breaks) protocols, and what's actually known from empirical research.*

10.1 Two governance layers

1. **Protocol governance** — chain itself (Bitcoin BIPs, Ethereum EIPs).
2. **Application governance** — on-chain DAOs (MakerDAO, Compound, Uniswap, Arbitrum DAO).

The mechanisms and incentive structures look superficially similar but operate very differently.

10.2 Bitcoin's governance — "rough consensus"

- BIPs (Bitcoin Improvement Proposals) → reference implementation → miners signal → users decide via running software (UASF).
- 2017 SegWit / Big Block war showed that **users**, not miners, are the ultimate authority — bitcoin (BTC) followed the user-activated soft fork while Bitcoin Cash split off.

The lesson: in PoW, **hashrate is hired, not sovereign**.

10.3 Ethereum's governance — coordinated chaos

- **EIPs** (Ethereum Improvement Proposals) drafted by core devs.
- **All Core Devs Calls** (ACD) — weekly meeting, no formal vote.
- **Client diversity** — Geth, Nethermind, Erigon, Besu, Reth — implementations must agree or chain forks.
- **Validators / users** ratify by running upgraded clients.

This loose process has produced the smoothest L1 upgrade cycle in the space (London, Shanghai, Cancun, Pectra, Fusaka).

10.4 DAO governance — the on-chain experiment

Mechanisms

Type	Example	Notes
Token-weighted voting	Compound, Uniswap	Plutocratic; whales dominate
Conviction voting	1Hive	Stake longer → more weight
Quadratic voting	Gitcoin Grants	Mitigates whales; Sybil-vulnerable
Reputation / soulbound	Coordinape, BrightID	Non-transferable

Optimistic / delegated	Optimism Citizens' House	Bicameral hybrid
veToken	Curve, Velodrome	Lock tokens to gain voting + boost

The veToken / bribe model

CRV locked → veCRV → vote weight on gauges → directs CRV emissions → projects bribe veCRV holders for emissions → secondary market on vlCVX (Convex). This is the **Curve Wars** — a real, multi-billion-dollar incentive market.

10.5 Tokenomics as a discipline

A protocol's token simultaneously plays three roles:

1. **Security** — bonded stake for PoS, fees for L1s, slashing collateral.
2. **Governance** — voting rights.
3. **Cash flow** (sometimes) — buybacks, fee dividends.

Token supply schedules

Pattern	Example
Fixed cap, halving	Bitcoin (21M)
Fixed cap, vesting	Most VC tokens (cliff + linear)
Inflationary	Ethereum issuance, Cosmos
Burn-deflationary	EIP-1559 (Ethereum), BNB
Rebasing	OHM, AMPL
Bond-curve / continuous	Olympus, Fei (RIP)

Vesting cliff "unlock walls"

Most VC-funded tokens unlock 6–36 months after TGE. Look at **Token Unlocks** or **TokenTerminal** to spot incoming sell pressure.

10.6 Real fundamentals: fees & revenue

After 2021's hype, the industry shifted to real revenue analysis:

Metric	What it means
Fees	Total paid by users
Revenue	Fees minus distributions back to users (e.g., to LPs)
PE-style ratios	FDV / annualized revenue

Take rate	Protocol's share of total fees

Tools: **Token Terminal, DefiLlama, Dune Analytics, Artemis**.

10.7 Governance attacks

Beanstalk (2022)

Attacker took a $1B flash loan, used it to vote in a malicious proposal that drained the treasury. Lesson: **time-lock all proposals**; never allow flash-loaned governance.

Tornado Cash governance (2023)

Attacker proposed an "upgrade" with a hidden malicious payload, passed it, took control. Lesson: review every line of every proposal, including innocuous-looking ones.

Quorum / voter apathy

Most DAOs see <5% voter turnout. A motivated minority can dominate. Mitigations: delegation (Compound's COMP delegates), proxy advisors (Tally), executive committees.

10.8 Mechanism design themes

Mechanism	Use
Vickrey auctions	NFT mints (rarely used yet)
Harberger taxes	Radical Markets, partial-common-ownership land
Quadratic funding	Public goods (Gitcoin)
Curation markets	Token-curated registries (TCRs)
Conviction voting	Long-term decisions

Most live DAOs still use simple token-weighted voting because anything else is harder to explain to participants.

10.9 Public goods funding

Open problem: how do you fund clients, libraries, audits — the underfunded layer of every blockchain?

- **Optimism RetroPGF** — retroactive payouts to ecosystem builders.
- **Gitcoin Grants** — quadratic funding rounds.
- **Protocol Guild** — Ethereum core dev vesting from app projects.
- **EIP-1559** — burns fees, distributes value indirectly to holders, not to builders.

10.10 Regulation

Jurisdiction	2026 stance (illustrative)
US	Patchwork; CFTC/SEC turf battle; ETH ETFs approved 2024
EU	MiCA in force since 2024
UK	Phased FCA framework
Singapore, UAE, HK	Active licensing regimes
China	Trading banned; CBDC heavy

Effects on protocols:

- KYC requirements for fiat ramps and stablecoin issuers.
- Sanctions screening (Tornado Cash sanctions, 2022).
- "Decentralization sufficiency" — ongoing legal test.

10.11 Reading list

- Buterin — *Notes on Blockchain Governance*, 2017.
- Werner et al. — *SoK: Decentralized Finance (DeFi)*, AFT 2022 (governance section).
- Posner & Weyl — *Radical Markets*, Princeton UP, 2018 (quadratic voting, Harberger).
- Buterin, Hitzig, Weyl — *A Flexible Design for Funding Public Goods*, 2019.
- Fritsch, Müller, Wattenhofer — *Analyzing Voting Power in Decentralized Governance*, 2022.
- *MakerDAO Whitepaper*; *Uniswap Governance docs*.

10.12 Lab (2 hrs)

1. Browse Tally.xyz. Pick one active proposal in a top-5 DAO; read it end-to-end.
2. Plot a token's circulating supply over time + scheduled unlocks (use Token Unlocks data).
3. Compute Compound's PE = FDV / annualized revenue from DefiLlama. Compare to Aave, Uniswap.

10.13 Quiz

1. Why is the Bitcoin SegWit/BCH split evidence that hashrate isn't sovereign?
2. State one weakness of token-weighted voting and one of quadratic voting.
3. Explain the veToken bribe market in one paragraph.
4. What did Beanstalk teach about flash-loan governance defense?
5. Why might fees ≠ revenue for an AMM?

Blockchain Module 11 — Advanced Consensus (DAGs, Sharding, HotStuff Family)

> ***Goal**: Move from textbook BFT to the 2023–2026 research frontier. After this module you can read SOSP/NSDI/PODC papers on consensus without translation.*

11.1 The performance ceiling of leader-based BFT

Leader-based protocols (PBFT, HotStuff, Tendermint) have a fundamental ceiling:

```
throughput ≤ leader_bandwidth / block_size
```

Because **one node** must disseminate the block per round. The 2020–2024 wave of DAG protocols breaks this by decoupling **transaction dissemination** from **ordering**.

11.2 DAG-based consensus

Narwhal & Bullshark (Facebook/Meta, then Sui/Aptos lineage)

- **Narwhal**: reliable, scalable mempool. Every validator runs its own "worker" lanes broadcasting batches. Produces a DAG of certified batches.
- **Bullshark**: a *zero-message-overhead* consensus on top of Narwhal. Total ordering is derived deterministically from the DAG structure.

Benchmarks: 130k+ TPS, sub-2-second finality.

Mysticeti (Sui, 2024)

Single-round commit, multi-leader DAG. Targets 300k+ TPS.

Aleph BFT (Aleph Zero)

Asynchronous DAG protocol, sub-second finality, randomized leader.

IOTA Tangle (historical)

A DAG where each new transaction validates two prior ones. Pioneered the DAG idea but had centralization issues (the Coordinator) that took years to resolve.

11.3 Sharding

Split state and execution across `K` parallel chains.

Challenges

Problem	Description
Cross-shard communication	Atomic txs across shards require commit protocols
State fragmentation	A user's state may live across shards
Validator security	Smaller shards = easier to corrupt
Data availability	Each shard's data must be retrievable by anyone

Approaches

Project	Sharding flavor
Ethereum (original plan, then abandoned execution sharding)	64 shards
Ethereum (current)	**Data sharding only** (danksharding) — execution off-loaded to L2s
Near	Nightshade — dynamic resharding
Polkadot	Shared security via relay chain; parachains are heterogeneous shards
Cosmos	App-chain sovereignty + IBC; not "sharding" in the classic sense

Ethereum's pivot: instead of execution sharding, do **data sharding** so rollups can scale execution. The "rollup-centric roadmap".

11.4 HotStuff lineage

Classic HotStuff (Yin et al., 2019)

Three-phase chained voting → linear leader rotation.

LibraBFT / DiemBFT → Aptos (AptosBFT v4)

Extends HotStuff with reputation-based leader selection and pipelined sub-second finality.

Jolteon / Ditto (Meta)

Merges HotStuff happy-path linearity with PBFT's view-change for adversarial scenarios.

Carousel, Bullshark, Tusk, Shoal

Each iteration squeezes another constant factor out.

11.5 Asynchronous protocols

If you don't want to assume **any** synchrony, you need randomness.

Protocol	Trick
HoneyBadger BFT (Miller et al., 2016)	Threshold encryption + atomic broadcast
Dumbo	Reduces HBBFT round count
Tusk	Async DAG on top of Narwhal

Async protocols are slower in good conditions but never wedge.

11.6 Restaking & shared security as a consensus pattern

EigenLayer (2023) introduces a new design pattern: validators on chain A also validate AVSs (Actively Validated Services). The AVS's safety budget = restaked ETH at risk.

This blurs the line between consensus, security, and the application layer — a 2024–2026 research area.

Risks: AVS slashing definitions are heterogeneous; correlated failures across AVSs; centralization through restaking aggregators.

11.7 MEV-aware consensus

Active research:

- **Crypto-economic encrypted mempools** (Shutter, F3B, Themis)
- **Order-fairness consensus** (Aequitas, Themis, Pompê)
- **Proposer-Builder Separation (PBS)** with in-protocol enshrinement (ePBS, on Ethereum roadmap)
- **Inclusion lists** — proposer can force certain txs into the next block to defeat censorship

---

11.8 Cross-chain consensus

Approach	Example
Light client bridges	IBC (Cosmos), zkBridge
Shared sequencers	Espresso, Astria
Shared validity proving	Polygon AggLayer, EigenDA + Espresso
Universal settlement	Polygon Pos→AggLayer; OP Superchain shared bridge

The 2024–2026 trend: minimize trust through ZK light clients; minimize fragmentation through shared infrastructure.

11.9 Reading list

- Yin et al. — *HotStuff: BFT consensus with linearity and responsiveness*, PODC 2019.

- Danezis et al. — *Narwhal & Tusk: A DAG-based Mempool and Efficient BFT Consensus*, EuroSys 2022.
- Spiegelman, Giridharan, Sonnino, Kokoris-Kogias — *Bullshark*, CCS 2022.
- Miller et al. — *The Honey Badger of BFT Protocols*, CCS 2016.
- Kelkar et al. — *Order-Fair Consensus*, CRYPTO 2020.
- Buterin — *Endgame* + *PBS roadmap* posts.

11.10 Lab (2 hrs)

Pick one Bullshark / Mysticeti benchmark paper. Reproduce the throughput-vs-latency plot in a Python simulator (not actually networked — just simulate the message exchange). Discuss what assumptions made the original 130k TPS achievable.

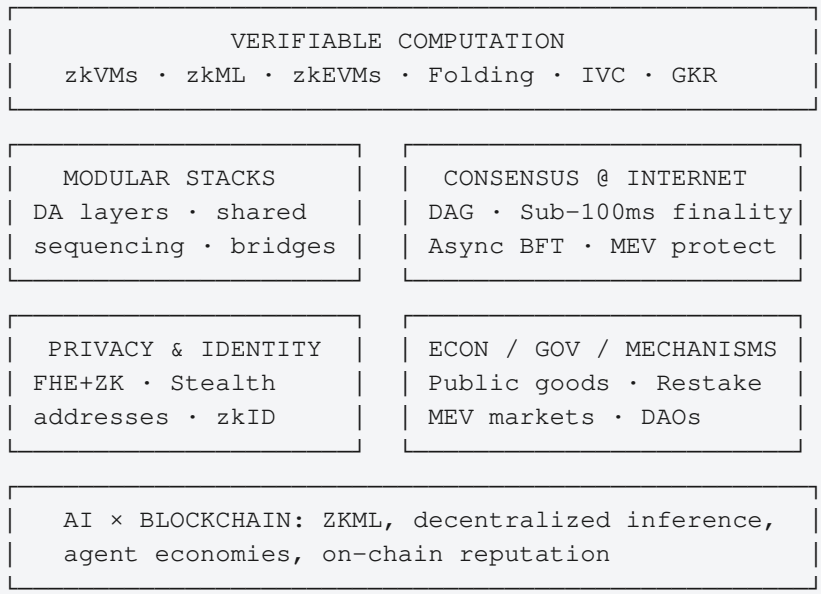
11.11 Quiz

1. Why do DAG-based protocols outperform leader-based BFT in throughput?
2. What did Ethereum trade away when it dropped execution sharding for data sharding?
3. State one tradeoff of asynchronous BFT protocols vs partially synchronous ones.
4. Describe one risk of EigenLayer's shared security model.
5. Why does MEV motivate consensus-level redesign and not just app-level fixes?

Blockchain Module 12 — Research Frontiers (2025–2027)

> ***Goal**: Identify what's actually new, where the citations are clustering, and which lines of work could plausibly become a Master's or PhD thesis.*

12.1 Frontier map



12.2 Top open research questions

1. Practical zkML at scale

- Can we prove inference for >1B parameter LLMs in seconds with sub-$1 cost?
- New: GKR-based proving (sumcheck), lookup arguments, folding schemes (Nova/HyperNova).

2. Stateless / minimally-stateful clients

- Verkle trees, history expiry (EIP-4444), light clients with full security.
- Ethereum's "The Verge" milestone.

3. Folding schemes & IVC

- Nova, SuperNova, HyperNova, ProtoStar, CycleFold.
- Goal: incrementally verifiable computation with O(1) prover overhead per step.

4. Encrypted mempools and MEV defense

- Threshold encryption (Shutter), FHE-based, time-lock encryption.
- Tradeoffs: latency, liveness under failure.

5. Sovereign vs shared sequencing

- Single rollup sequencer = censorship risk.
- Shared sequencers (Espresso, Astria) trade composability for centralization risk.
- Active design space.

6. Account abstraction beyond ERC-4337

- Native AA (EIP-7702 path), recovery, MFA, social recovery, biometric signing.

7. Restaking risk modeling

- Correlated slashing across AVSs.
- What's the right capital-budget for slashable bonds across services?

8. Privacy-preserving compliance

- "Selective disclosure" — prove you're not OFAC-sanctioned without revealing identity.
- Aztec, Zcash deposit-screening, Sismo Connect.

9. Long-range and quantum resistance

- Post-quantum signature migration paths.
- Hash-based signatures (XMSS, SPHINCS+) and lattice schemes (CRYSTALS-Dilithium).

10. On-chain games & autonomous worlds

- Dark Forest (zk fog of war), MUD, Argus — game state as smart contract.
- Pushes ZK proving and gas optimization to extremes.

11. Decentralized AI infrastructure

- Compute marketplaces (Akash, Render, io.net).
- Model marketplaces and reputation (Bittensor, Ritual).
- See intersection module.

12. Formal verification at scale

- Specs auto-generated from upgrades; ML-assisted invariant inference.

12.3 Venues to track

Venue	Tracks
IEEE S&P, USENIX Security, CCS, NDSS	Security
CRYPTO, EUROCRYPT, ASIACRYPT, TCC	Cryptography & ZK
PODC, DISC, OSDI, SOSP, NSDI	Distributed systems
AFT (Advances in Financial Technologies)	DeFi, mechanism design

FC (Financial Cryptography)	Applied crypto + DeFi
DSN	Dependability + BFT
NeurIPS / ICML / ICLR	ZKML, decentralized ML
ACM EC, WINE	Mechanism design, governance

Workshops worth submitting to as a first paper:

- DeFi @ CCS, ConsensusDay @ ESORICS, ZKProof workshop, Decentralizing Finance @ FC, FROST.

12.4 Toolchains worth building on

Tool	Why
Foundry (Solidity dev)	Industry standard test/fuzz framework
Risc Zero, SP1, Jolt	zkVMs — prove any Rust program
Circom, Noir, Halo2	Hand-written circuits
EigenLayer SDK	AVS development
Lambdaclass plonky3	Modern STARK toolkit
Hardhat / Tenderly	Mainstream Solidity tooling

12.5 How to pick a thesis-scale topic

1. Pick one of the 12 open questions above.
2. Read the **5 most-cited papers** of the last 18 months in that subarea.
3. Find a **clean primitive missing** or a **clean attack class undeveloped**.
4. Build the smallest possible empirical artifact (a benchmark, a measurement study, a circuit).
5. Submit to a workshop first. Iterate to a full venue.

The fastest first-paper recipe in blockchain research: **a measurement paper**. Empirical chain data is abundant; few researchers can both query archives and reason about protocol semantics.

12.6 Lab / capstone (open-ended)

Pick **one** of:

- Build a working zkML pipeline that proves an MNIST CNN forward pass with EZKL or Risc Zero. Measure proof time and verifier gas cost.
- Reproduce a benchmark from the Narwhal-Bullshark paper using their open-source code.
- Conduct a measurement of MEV-Boost relay behavior over 1 month of Ethereum blocks.
- Implement a stealth address scheme (EIP-5564) and write a usability analysis.

12.7 Quiz

1. Name two recent folding schemes and what they improve.
2. What does Ethereum's "The Verge" milestone aim for?
3. Why might shared sequencing recentralize what rollups decentralized?
4. State one open question in zkML at scale.
5. Why is *measurement* often the easiest first paper for a new researcher?

Appendix

Appendix A — Blockchain × XAI Research Intersection

The chapter that follows links this subject to its sister discipline, surveying the small but active research area where the two converge.

Intersection — Verifiable, Explainable, and Decentralized AI

> ***Goal**: This is the research-value module. Where Blockchain meets XAI, several emerging research lines need contributors. A Master's thesis here has unusually high impact-per-effort ratio.*

I.1 Why these two fields converge

Blockchain solves: **verifiability and integrity of computation in adversarial environments.** XAI solves: **understanding and trust of model behavior.**

Together, they tackle the question:

> *Can we know — and prove — what an AI did, why it did it, and that no one tampered with it?*

Three convergence areas:

1. **Verifiable AI** — cryptographic proofs that a model output is correct (ZKML).
2. **Decentralized AI** — model training/inference distributed and incentivized via tokens.
3. **Auditable / Explainable AI** — on-chain logs and explanations for regulatory compliance.

I.2 ZKML — Zero-Knowledge Machine Learning

The problem

A model owner can prove to a verifier that:

- "This output `y` is the result of running model `M` on input `x`."

Optionally hiding:

- The model weights (IP protection)
- The input (privacy)
- Both

Why hard

Neural networks use:

- **Floating-point arithmetic** — ZK circuits work over finite fields → quantization required.
- **Non-linear ops** (ReLU, GELU, softmax) — expensive to express in arithmetic constraints.
- **Large parameter counts** — billions of multiplications → billions of constraints.

Active toolchains (2024–2026)

Tool	Approach

EZKL	Halo2 backend, ONNX-friendly. Quantization + lookup-arg optimizations
Modulus Labs (Remainder)	GKR / sum-check for matrix multiplications
Risc Zero zkVM	General-purpose zkVM, run PyTorch in Rust via tract
Giza	StarkNet + Cairo for on-chain inference
DDKang's zkml	Halo2, focused on ResNet-scale
Inference Labs	Production ZK inference SaaS
Worldcoin	Uses semaphores + ZK for proof of personhood

State of the art

- ResNet-50 inference: provable in ~minutes on modern hardware, ~MB proofs.
- Small CNN MNIST: seconds, KB proofs.
- LLMs (>1B parameters): largely impractical end-to-end today; partial proofs possible.

Why it matters for XAI

A ZK proof of *correct inference* is necessary but not sufficient for trust. The next step: **ZK proofs of explanation**. Examples:

- Prove that a SHAP value reported externally was correctly computed from the model.
- Prove that a counterfactual explanation is genuinely the minimum-cost one.
- Prove that an attention-based "reason" was correctly extracted.

This is **mostly open research as of 2026** — there's room for early entrants.

I.3 On-chain machine learning

Approaches

Approach	Trust assumption
Inference **off-chain**, post **proof on-chain**	ZK or optimistic
Inference **on-chain** in a smart contract	High gas; bounded models only
Inference in **TEE** with attestation	Hardware trust (Intel SGX, AMD SEV)
Inference via **MPC**	Multiple non-colluding parties
Inference with **FHE**	Pure cryptography, very expensive

EVM-based on-chain ML (Modulus Labs' Ocean, ORA) typically posts ZK proofs of small models for verifiable randomness, oracle aggregation, content moderation triggers, etc.

I.4 Decentralized AI infrastructure

Compute marketplaces

Project	Layer
Akash	General Kubernetes compute
Render	GPU rendering + ML inference
io.net	Distributed GPU cluster
Gensyn	Distributed training with verification
Bittensor	Subnet-based incentivized model marketplace
Ritual	AI inference oracle for smart contracts

Trust model varies — some use ZK, some use challenge-response / fraud proofs.

Model marketplaces & reputation

Open problems:

- How do you verify a model's claimed performance?
- How do you reward good models without enabling sybils?
- How do you handle adversarial submissions (model poisoning)?

Bittensor's "Yuma consensus" weights subnet contributions by peer voting — a research-grade mechanism design effort.

I.5 Federated learning + blockchain

Federated learning (FL) trains a model across many devices without centralizing data. Blockchains can:

- Coordinate aggregation rounds.
- Reward participants by stake / contribution.
- Audit updates for poisoning.
- Verify aggregation via ZK / MPC.

Research papers stack: FL + blockchain + differential privacy + XAI.

Trust model: typically a permissioned consortium chain (Hyperledger Fabric, IBC chains). Honest-majority assumption.

I.6 Decentralized identity and AI

Tool	Use
W3C DIDs / Verifiable Credentials	Self-sovereign identity
Soulbound tokens (SBT)	Non-transferable reputation

zkID / Sismo / Worldcoin	Privacy-preserving "uniqueness" proofs
Polygon ID	Selective disclosure of attributes

Why for AI: deepfake / agentic-AI era requires **proof of personhood** to distinguish humans from agents. Worldcoin's iris-scan-based PoP is the most controversial deployment.

I.7 Audit trails: explanations on-chain

For regulatory compliance, you may want:

```
inputs_hash + model_hash + outputs_hash + explanation_hash → on-chain commitment
```

Anyone can later verify the audit trail. Variants:

- **Off-chain explanation, on-chain hash** (cheap, requires data availability).
- **On-chain explanation** (expensive, only for small models or critical decisions).
- **ZK explanation** (prove the explanation was correctly computed without revealing input or model).

Use cases:

- Credit decision explanations stored immutably for regulator audit.
- Healthcare AI outputs with retroactive verifiability.
- Election counting AI with public auditability.

I.8 Adversarial concerns

This intersection inherits **both** fields' attack surfaces:

From blockchain	From XAI
51% attack on verification chain	Adversarial explanations (Slack et al.)
Bridge hacks (cross-chain proofs)	Fairwashing
Oracle manipulation	Manipulated saliency
MEV on inference auctions	CoT unfaithfulness
Smart contract bugs in verifier	Proof-system soundness bugs

New combined risks:

- **Adversarial ZK proofs** — soundness errors in custom circuits.
- **Quantization attacks** — exploit the fp→fixed conversion to skew explanations.
- **Replay of stale model commitments**.
- **Sybil attacks on decentralized model voting**.

I.9 Open research questions

1. **Sub-second ZK proofs of small NN inference + SHAP** — currently minutes; gap is 100×.
2. **Faithful, provable LLM explanations** — combining mech-interp with ZK.
3. **Decentralized model training with verifiable convergence** — beyond FedAvg + Merkle commits.
4. **Privacy-preserving fairness audits** — prove fairness without revealing individual records.
5. **Soulbound reputation for model providers** with non-gameable scoring.
6. **Cross-chain AI oracles** — AI predictions consumed across multiple L1s.
7. **MEV-protected AI inference markets**.
8. **Compliance-grade audit trails** for EU AI Act, with ZK for trade-secret protection.
9. **On-chain explanation registries** that survive model versioning.
10. **Watermarking AI outputs** with blockchain-anchored provenance (deepfake defense).

I.10 Sample research project (Master's-scale)

Title: "Verifiable Feature Attribution for Black-Box Inference: A ZK-SHAP Prototype"

Plan:

1. Train a small (1–5M param) tabular model.
2. Implement KernelSHAP in a ZK-friendly form (fixed-point, batch evaluation).
3. Compile to EZKL / Risc Zero.
4. Generate proofs for ~10 test inputs; measure proof size, prover time, verifier gas.
5. Test soundness via adversarial model (Slack attack adapted) — does the proof catch it?
6. Write up as a workshop paper (target: AFT, IEEE Blockchain, BlackBoxNLP, or zkProof workshop).

This is a clean, novel intersection topic with a 3–6 month timeline.

I.11 Reading list

- Kang, Hashimoto, Stoica, Sun — *zkML and verifiable inference* (various 2023–2024 papers).
- Lee, Kang, Lee, Boneh — *EZKL and Halo2 ZK Inference*, 2023.
- Feng, Yan, Zhao, Lai, Lin — *Cerberus: ZK-SNARK-based Verifiable ML Inference*, 2021.
- Liu, Xie, Zhang — *zkCNN: Zero-Knowledge Proofs for Convolutional Neural Networks*, CCS 2021.
- Modulus Labs — *Cost of Intelligence* benchmark.
- Bittensor whitepaper.
- McMahan et al. — *Communication-Efficient Learning of Deep Networks from Decentralized Data (FedAvg)*, AISTATS 2017.
- Bonawitz et al. — *Practical Secure Aggregation for Privacy-Preserving ML*, CCS 2017.
- Karimi et al. — *Verifiable Federated Learning via ZK proofs*, various 2023.
- W3C — *DID Specification*; *Verifiable Credentials Data Model*.
- *EU AI Act* — Articles on high-risk systems and transparency.

I.12 Lab (5 hrs)

1. Install EZKL.
2. Train a 2-layer MLP on MNIST in PyTorch; export to ONNX.
3. Run `ezkl gen-settings`, `compile-circuit`, `setup`, `gen-witness`, `prove`, `verify`.

4. Time the proving step; measure proof size.
5. Compute a SHAP explanation off-chain; commit its hash on Sepolia.

I.13 Quiz

1. Why are ReLU and softmax expensive in ZK circuits?
2. State three trust models for on-chain ML.
3. How would you prove "this SHAP value was correctly computed" without revealing the model?
4. Why does Bittensor's Yuma consensus need a mechanism-design contribution to remain non-gameable?
5. Pick one of the 10 open research questions and write a 3-sentence research plan.